

Hashing in Data Structures and Algorithms: Techniques, Analysis, and Applications

Priyanshu Shekhar Choudhury

Department of Computer Science, Lovely Professional University

Abstract

Hashing is a pivotal concept in data structures and algorithms that facilitates efficient data storage and retrieval with constant-time average performance. This paper presents a comprehensive overview of hash functions, hash tables, and collision resolution strategies, including separate chaining, linear probing, quadratic probing, and double hashing. Performance characteristics are analyzed in terms of load factor and time complexity. The paper concludes with a discussion of practical applications of hashing in modern computing environments, including database indexing, caching, and distributed systems.

Keywords—hashing; hash table; collision resolution; data structures; algorithms; open addressing; separate chaining

I. INTRODUCTION

Hashing is one of the most fundamental techniques in Data Structures and Algorithms (DSA). It provides an efficient mechanism for storing and retrieving data in constant average time, $O(1)$, making it indispensable in modern computing systems. From database indexing to cryptographic applications, hashing underpins a wide range of real-world solutions.

A hash function maps input data of arbitrary size to a fixed-size value, commonly referred to as a hash code or digest. The resulting value serves as an index into a hash table, enabling rapid data access without linear search. This paper explores the principles of hashing, the structure of hash tables, common collision resolution techniques, and performance considerations.

II. HASH FUNCTIONS

A hash function $h(k)$ maps a key k to an integer index within the range $[0, m-1]$, where m is the size of the hash table. An ideal hash function satisfies the following properties:

- **Determinism:** The same key always produces the same hash value.
- **Uniformity:** Keys are distributed evenly across the table.
- **Efficiency:** Computation of the hash value must be fast.
- **Low Collision Rate:** Different keys should rarely map to the same index.

Common hash functions include the Division Method, $h(k) = k \bmod m$, and the Multiplication

IV. COLLISION RESOLUTION

A collision occurs when two different keys hash to the same index. Since collisions are inevitable by the pigeonhole principle, efficient resolution strategies are essential.

A. Separate Chaining

Each bucket contains a linked list of all key-value pairs that hash to that index. Insertion is $O(1)$ and search is $O(1 + \alpha)$ on average. This approach handles high load factors gracefully but incurs memory overhead due to pointer storage.

B. Open Addressing

All elements are stored within the hash table itself. When a collision occurs, the algorithm probes for the next available slot using one of the following strategies:

- **Linear Probing:** $h(k, i) = (h(k) + i) \bmod m$. Simple but prone to primary clustering.
- **Quadratic Probing:** $h(k, i) = (h(k) + c_1i + c_2i^2) \bmod m$. Reduces clustering at the cost of complexity.
- **Double Hashing:** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Provides near-uniform distribution and minimizes clustering.

V. PERFORMANCE ANALYSIS

Table I summarizes the average and worst-case time complexities for hash table operations under both chaining and open addressing schemes.

Operation	Average	Worst
Insert	$O(1)$	$O(n)$
Search	$O(1)$	$O(n)$

Method, $h(k) = \text{floor}(m * (k * A \bmod 1))$, where A is a constant (typically 0.6180339887). Polynomial rolling hash functions are commonly used for string keys, expressed as $h(s) = \sum s[i] * p^i \bmod M$.

III. HASH TABLES

A hash table is a data structure that implements an associative array (key-value pairs) using a hash function to compute indices. Each slot in the table is called a bucket. Hash tables support three primary operations:

- **Insert(k, v):** Compute $h(k)$ and store value v at that index.
- **Search(k):** Compute $h(k)$ and retrieve the value stored there.
- **Delete(k):** Compute $h(k)$ and remove the entry from that slot.

The load factor $\alpha = n/m$, where n is the number of elements and m is the table size, is a critical metric. As α increases, performance degrades. It is common practice to resize the table (rehashing) when α exceeds a threshold such as 0.75.

Delete	$O(1)$	$O(n)$
--------	--------	--------

TABLE I. Hash Table Time Complexity

VI. APPLICATIONS

Hashing is used extensively in: database indexing (B+ trees and hash indexes), compiler symbol tables, caches (CPU cache, DNS cache), set and map data structures in languages like Python and Java, cryptographic hash functions (SHA-256, MD5), and distributed systems (consistent hashing for load balancing).